UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2023 Algoritmos y Estructuras de Datos
Implementación de AVL Tree
Semana 9

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

6 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo de un AVL: guardar la altura y mantenerla en $O(1)$

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo de un AVL: guardar la altura y mantenerla en $O(1)$
- Las rotaciones como reconexión de punteros

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo de un AVL: guardar la altura y mantenerla en $O(1)$
- Las rotaciones como reconexión de punteros
- INSERTAR y ELIMINAR: los del BST, más rebalancear al volver de la recursión

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El nodo de un AVL: guardar la altura y mantenerla en $O(1)$
- Las rotaciones como reconexión de punteros
- Insertar y Eliminar: los del BST, más rebalancear al volver de la recursión
- Seguir inserciones y eliminaciones paso a paso, y probar que el árbol siga siendo un AVL

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada

Vimos que $|FB| \leq 1$ en todo nodo garantiza altura $O(\lg n)$, y que una o dos rotaciones restauran el balance tras insertar.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada

Vimos que $|FB| \leq 1$ en todo nodo garantiza altura $O(\lg n)$, y que una o dos rotaciones restauran el balance tras insertar.

Para decidir si rota, cada nodo necesita su factor de balance. ¿Cómo lo sabe sin recorrer todo su subárbol?

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada

Vimos que $|FB| \leq 1$ en todo nodo garantiza altura $O(\lg n)$, y que una o dos rotaciones restauran el balance tras insertar.

Para decidir si rota, cada nodo necesita su factor de balance. ¿Cómo lo sabe sin recorrer todo su subárbol?

Guardando su altura

Si cada nodo guarda su altura, el factor de balance sale de mirar solo a sus dos hijos. Todo lo de hoy se apoya en esa idea: guardar en cada nodo un dato que se calcula con sus hijos.

El nodo

El nodo de un AVL

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Campos

- $v.clave$, $v.izq$, $v.der$: como en el BST.
- $v.altura$: la altura del subárbol de v.

El nodo de un AVL

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Campos

- $v.clave$, $v.izq$, $v.der$: como en el BST.
- $v.altura$: la altura del subárbol de v.

En el BST clases anteriores también guardábamos $v.padre$. ¿Aquí hace falta?

El nodo de un AVL

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Campos

- *v.clave*, *v.izq*, *v.der*: como en el BST.
- *v.altura*: la altura del subárbol de *v*.

En el BST clases anteriores también guardábamos *v.padre*. ¿Aquí hace falta?

No, si trabajamos de forma recursiva

Cada función recibe la raíz de un subárbol y **devuelve** la nueva raíz de ese subárbol (puede cambiar si hubo rotación). Quien llamó cuelga lo devuelto en su lugar: $v.izq \leftarrow \textsc{Insertar}(v.izq, k)$. Y la raíz del árbol se actualiza igual: $T.raiz \leftarrow \textsc{Insertar}(T.raiz, k)$.

Tres funciones auxiliares

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 1: $h(v)$

si $v = \text{NULO}$ **entonces devolver** -1
devolver $v.altura$

Tres funciones auxiliares

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 4: $h(v)$

si $v = \text{NULO}$ **entonces devolver** -1
devolver $v.altura$

Algoritmo 5: $\text{ACTUALIZAR}(v)$

$v.altura \leftarrow 1 + \text{máx}\left(h(v.izq),\ h(v.der)\right)$

Tres funciones auxiliares

Algoritmo 7: $h(v)$

si $v = \text{NULO}$ **entonces devolver** -1
devolver $v.altura$

Algoritmo 8: $\text{ACTUALIZAR}(v)$

$v.altura \leftarrow 1 + \text{máx}\left(h(v.izq),\ h(v.der)\right)$

Algoritmo 9: $FB(v)$

devolver $h(v.izq) - h(v.der)$

Tres funciones auxiliares

Algoritmo 10: $h(v)$

si $v = \text{NULO}$ **entonces devolver** -1
devolver $v.altura$

Algoritmo 11: $\text{ACTUALIZAR}(v)$

$v.altura \leftarrow 1 + \text{máx}\left(h(v.izq),\ h(v.der)\right)$

Algoritmo 12: $FB(v)$

devolver $h(v.izq) - h(v.der)$

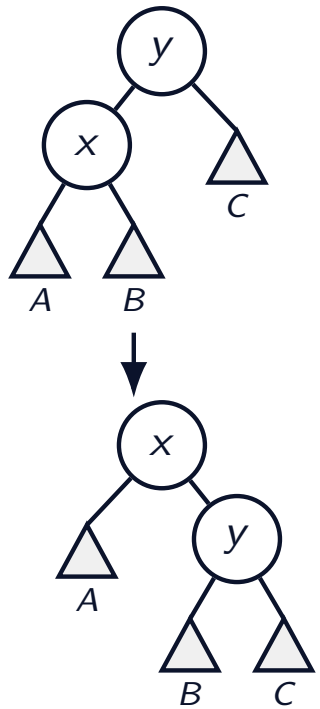
Son la h y el FB de la clase anterior (con $h(\text{NULO}) = -1$), y las tres cuestan $O(1)$: solo leen lo que ya guardaron los hijos. La condición es llamar a $\text{ACTUALIZAR}(v)$ cada vez que cambia un hijo de v.

Rotaciones

Rotar-Derecha

Algoritmo 13: ROTAR-DERECHA(y)

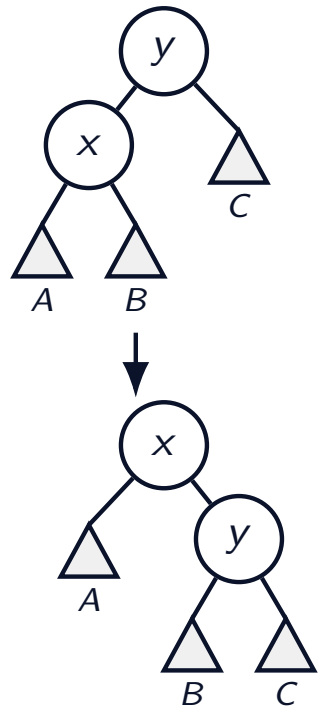
$x \leftarrow y.izq$
$y.izq \leftarrow x.der$
$x.der \leftarrow y$
ACTUALIZAR(y)
ACTUALIZAR(x)
devolver x



Rotar-Derecha

Algoritmo 14: ROTAR-DERECHA(y)

$x \leftarrow y.izq$
$y.izq \leftarrow x.der$
$x.der \leftarrow y$
ACTUALIZAR(y)
ACTUALIZAR(x)
devolver x



La línea 2 mueve el subárbol B: es el único que cambia de padre. El orden de las dos llamadas a ACTUALIZAR importa: y quedó abajo, así que va primero; la altura de x depende de la de y ya corregida.

Rotar-Izquierda

Algoritmo 15: ROTAR-IZQUIERDA(x)

$y \leftarrow x.der$
$x.der \leftarrow y.izq$
$y.izq \leftarrow x$
ACTUALIZAR(x)
ACTUALIZAR(y)
devolver y

Rotar-Izquierda

Algoritmo 16: ROTAR-IZQUIERDA(x)

$y \leftarrow x.der$
$x.der \leftarrow y.izq$
$y.izq \leftarrow x$
ACTUALIZAR(x)
ACTUALIZAR(y)
devolver y

El espejo exacto de ROTAR-DERECHA: *izq* y *der* intercambiados en cada línea. Las dos son $O(1)$: reconectan tres punteros, sin mirar dentro de A, B ni C.

Rebalancear: elegir el caso

Algoritmo 17: REBALANCEAR(v)

```
ACTUALIZAR(v)
si FB(v) > 1 entonces
    si FB(v.izq) < 0 entonces
        v.izq ← ROTAR-IZQUIERDA(v.izq)                    // LR
    devolver ROTAR-DERECHA(v)                             // LL
si FB(v) < −1 entonces
    si FB(v.der) > 0 entonces
        v.der ← ROTAR-DERECHA(v.der)                      // RL
    devolver ROTAR-IZQUIERDA(v)                           // RR
devolver v                                  // ya estaba balanceado
```

Insertar

Insertar

Algoritmo 18: INSERTAR(v, k)

si $v = \text{NULO}$ **entonces**
  devolver nuevo nodo con $clave = k$, $altura = 0$
si $k < v.clave$ **entonces**
  $v.izq \leftarrow$ INSERTAR$(v.izq, k)$
en otro caso
  $v.der \leftarrow$ INSERTAR$(v.der, k)$
devolver REBALANCEAR(v)

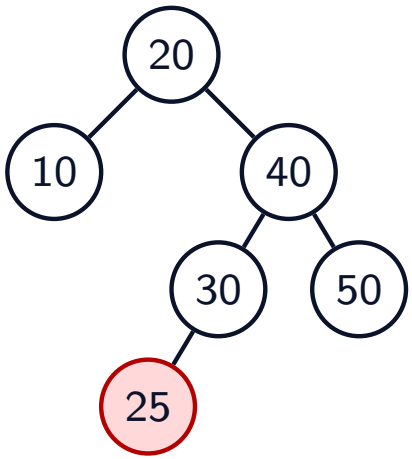
Insertar

Algoritmo 19: INSERTAR(v, k)

si $v =$ NULO **entonces**
  devolver nuevo nodo con $clave = k$, $altura = 0$
si $k < v.clave$ **entonces**
  $v.izq \leftarrow$ INSERTAR$(v.izq, k)$
en otro caso
  $v.der \leftarrow$ INSERTAR$(v.der, k)$
devolver REBALANCEAR(v)

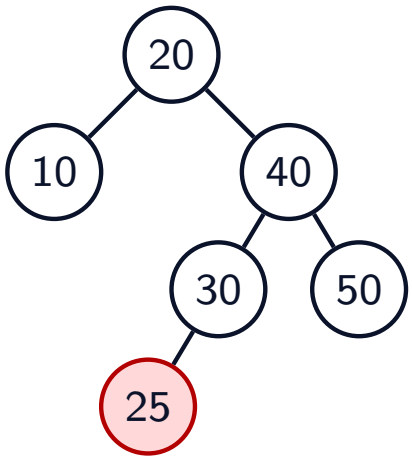
Es el INSERTAR del BST, escrito de forma recursiva, con una línea nueva al final. Al volver de la recursión se pasa por **todos** los ancestros del nodo nuevo, de abajo hacia arriba: justo los únicos cuya altura pudo cambiar.

Insertar, paso a paso



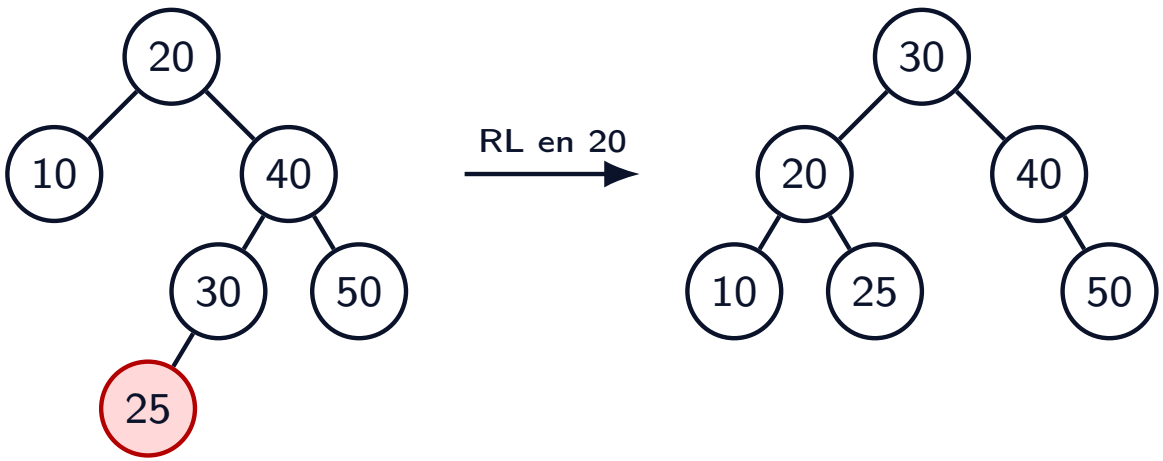
Vuelve a	altura	FB	Rebalancear hace...

Insertar, paso a paso



Vuelve a	altura	FB	Rebalancear hace...
30	1	+1	nada
40	2	+1	nada

Insertar, paso a paso



Vuelve a	altura	FB	Rebalancear hace...
30	1	$+1$	nada
40	2	$+1$	nada
20	3	-2	su hijo 40 tiene $FB = +1$: RL

¿Por qué basta con una sola rotación?

La recursión llama a REBALANCEAR en todos los ancestros. ¿Puede tener que rotar en más de uno?

¿Por qué basta con una sola rotación?

La recursión llama a REBALANCEAR en todos los ancestros. ¿Puede tener que rotar en más de uno?

No: la rotación devuelve la altura de antes

Si antes de insertar el subárbol del nodo desbalanceado tenía altura H, la inserción lo subió a $H + 1$ (por eso su FB llegó a ± 2). Tras la rotación, simple o doble, vuelve a medir H: desde arriba es como si nada hubiera cambiado.

¿Por qué basta con una sola rotación?

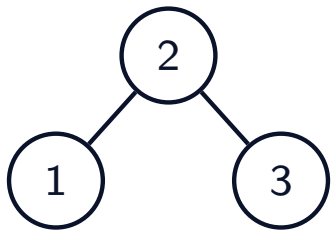
La recursión llama a REBALANCEAR en todos los ancestros. ¿Puede tener que rotar en más de uno?

No: la rotación devuelve la altura de antes

Si antes de insertar el subárbol del nodo desbalanceado tenía altura H, la inserción lo subió a $H + 1$ (por eso su FB llegó a ± 2). Tras la rotación, simple o doble, vuelve a medir H: desde arriba es como si nada hubiera cambiado.

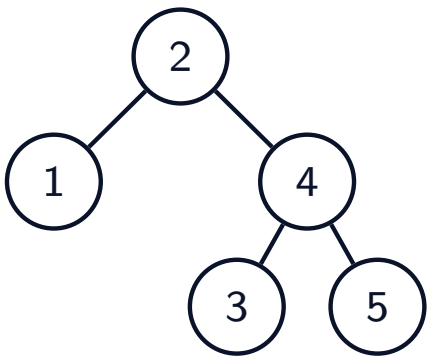
En el ejemplo, el subárbol de 20 medía 2 antes de insertar 25; después de la rotación, el de 30 vuelve a medir 2. INSERTAR es $O(\lg n)$: bajar, y a lo más una rotación (simple o doble) al volver.

Insertar $1, 2, \ldots, 7$ en orden



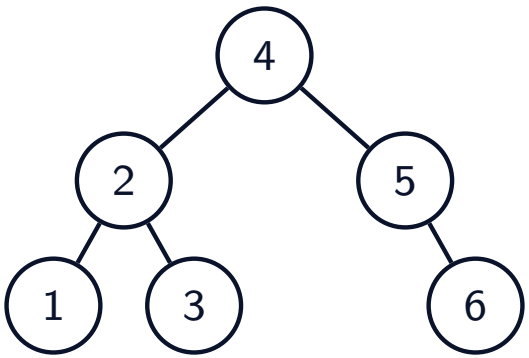
Al insertar 3, la raíz 1 queda con $FB = -2$ y su hijo 2 con -1: RR en 1.

Insertar $1, 2, \ldots, 7$ en orden



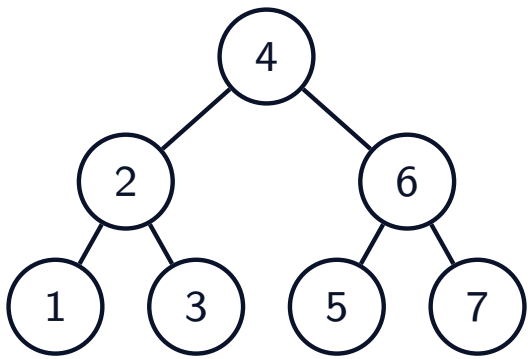
4 entra sin rotar; al insertar 5, 3 queda con $FB = -2$: RR en 3.

Insertar $1, 2, \ldots, 7$ en orden



Al insertar 6, el primer desbalanceado es la raíz 2 ($FB = -2$, su hijo 4 con -1): RR en 2.

Insertar $1, 2, \ldots, 7$ en orden



Al insertar 7, RR en 5: el árbol queda perfecto, de altura 2.

Un BST sin balancear terminaría como una cadena de altura 6

Es el peor caso de la clase de BST. El AVL rota cuatro veces, siempre con el caso RR, y termina con la menor altura posible.

Eliminar

Eliminar

Algoritmo 20: ELIMINAR(v, k)

```
si v = NULO entonces devolver NULO
si k < v.clave entonces
    v.izq ← ELIMINAR(v.izq, k)
si no, si k > v.clave entonces
    v.der ← ELIMINAR(v.der, k)
en otro caso
    si v.izq = NULO entonces devolver v.der
    si v.der = NULO entonces devolver v.izq
    s ← MÍNIMO(v.der)                          // el sucesor
    v.clave ← s.clave
    v.der ← ELIMINAR(v.der, s.clave)
devolver REBALANCEAR(v)
```

¿Por qué aquí sí puede rotar varias veces?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Al insertar, una rotación bastaba. ¿Qué cambia al eliminar?

¿Por qué aquí sí puede rotar varias veces?

Al insertar, una rotación bastaba. ¿Qué cambia al eliminar?

La rotación puede dejar el subárbol más bajo

Al eliminar, el subárbol del nodo desbalanceado había **bajado** de altura. La rotación lo balancea, pero puede dejarlo con altura menor a la que tenía antes de eliminar, y eso puede desbalancear a un ancestro más arriba.

¿Por qué aquí sí puede rotar varias veces?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

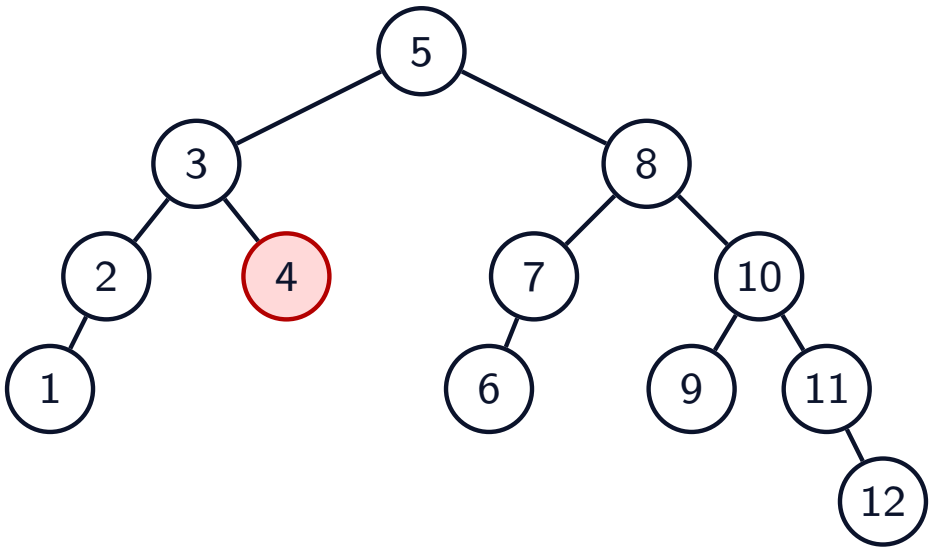
Al insertar, una rotación bastaba. ¿Qué cambia al eliminar?

La rotación puede dejar el subárbol más bajo

Al eliminar, el subárbol del nodo desbalanceado había **bajado** de altura. La rotación lo balancea, pero puede dejarlo con altura menor a la que tenía antes de eliminar, y eso puede desbalancear a un ancestro más arriba.

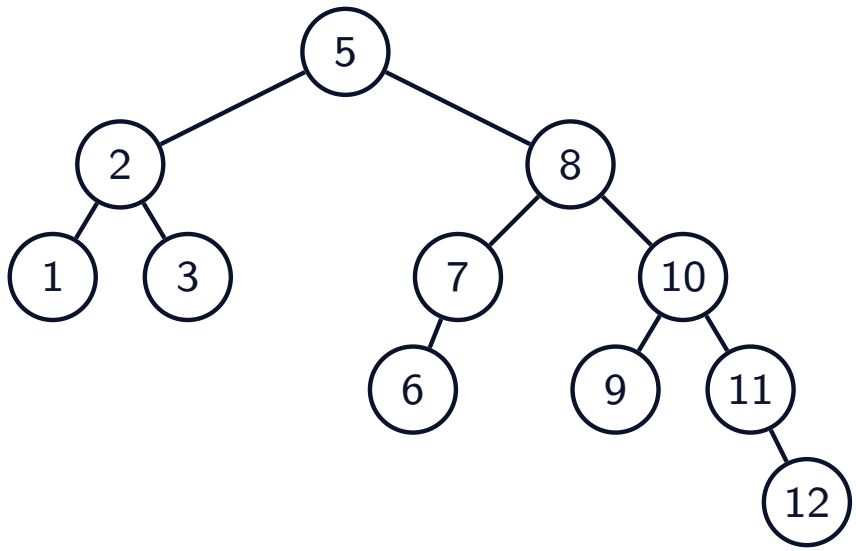
Por eso cada ancestro puede necesitar su propia rotación. Igual es $O(\lg n)$: a lo más una rotación (simple o doble) por cada nivel del camino.

Eliminar, paso a paso



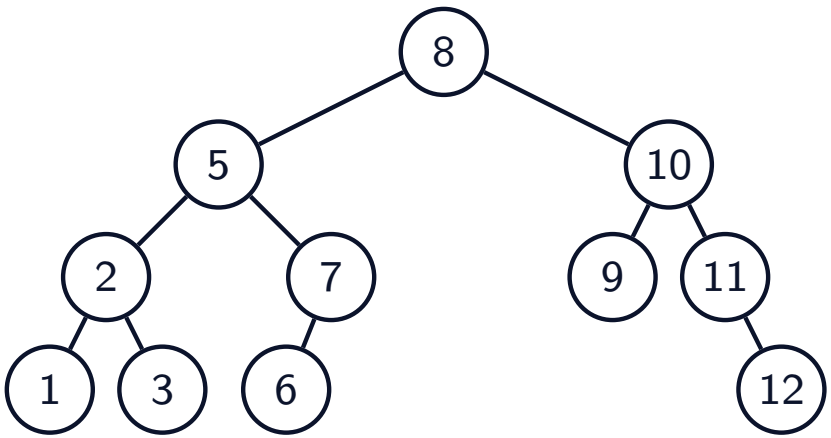
Se elimina la hoja 4 de este AVL de 12 claves.

Eliminar, paso a paso



Al volver a 3: $FB(3) = +2$ y su hijo 2 tiene $+1$: LL en 3. El subárbol pasa de altura 2 a 1.

Eliminar, paso a paso



Al volver a 5: $FB(5) = 1 - 3 = -2$ y su hijo 8 tiene -1: RR en 5. Dos rotaciones, en dos niveles.

La primera rotación arregló a 3 pero dejó su subárbol más bajo, y eso desbalanceó a 5: justo lo que al insertar no puede pasar.

Probar la implementación

¿El árbol sigue siendo un AVL?

Algoritmo 21: VERIFICAR(v, lo, hi) — devuelve $h(v)$, o FALLA

si $v = \text{NULO}$ **entonces devolver** -1
si ***no*** $lo < v.clave < hi$ **entonces devolver** FALLA
$a \leftarrow \text{VERIFICAR}(v.izq, lo, v.clave)$, $b \leftarrow \text{VERIFICAR}(v.der, v.clave, hi)$
si $a = \text{FALLA}$ ***o*** $b = \text{FALLA}$ **entonces devolver** FALLA
si $v.altura \neq 1 + \text{máx}(a, b)$ ***o*** $|a - b| > 1$ **entonces devolver** FALLA
devolver $v.altura$

¿El árbol sigue siendo un AVL?

Algoritmo 22: $\textsc{Verificar}(v, lo, hi)$ — devuelve $h(v)$, o $\textsc{falla}$

si $v = \textsc{nulo}$ **entonces devolver** -1
si ***no*** $lo < v.clave < hi$ **entonces devolver** $\textsc{falla}$
$a \leftarrow \textsc{Verificar}(v.izq, lo, v.clave)$, $b \leftarrow \textsc{Verificar}(v.der, v.clave, hi)$
si $a = \textsc{falla}$ ***o*** $b = \textsc{falla}$ **entonces devolver** $\textsc{falla}$
si $v.altura \neq 1 + \text{máx}(a, b)$ ***o*** $|a - b| > 1$ **entonces devolver** $\textsc{falla}$
devolver $v.altura$

Revisa las tres cosas que la implementación tiene que mantener: el orden de BST, la altura guardada en cada nodo y $|FB| \leq 1$. Se llama con $\textsc{Verificar}(T.raiz, -\infty, +\infty)$ y cuesta $O(n)$: sirve para probar, no para usar en cada operación.

Errores comunes

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Llamar a INSERTAR, ELIMINAR o a una rotación sin colgar lo que devuelven: $v.izq \leftarrow \text{INSERTAR}(v.izq, k)$, no solo $\text{INSERTAR}(v.izq, k)$.

Errores comunes

- Llamar a INSERTAR, ELIMINAR o a una rotación sin colgar lo que devuelven: $v.izq \leftarrow \text{INSERTAR}(v.izq, k)$, no solo $\text{INSERTAR}(v.izq, k)$.
- Actualizar x antes que y en ROTAR-DERECHA: la altura de x usa la de y, que todavía no se corrigió.
- Calcular $FB(v)$ en REBALANCEAR antes de $\text{ACTUALIZAR}(v)$.

Errores comunes

- Llamar a INSERTAR, ELIMINAR o a una rotación sin colgar lo que devuelven: $v.izq \leftarrow \text{INSERTAR}(v.izq, k)$, no solo $\text{INSERTAR}(v.izq, k)$.
- Actualizar x antes que y en ROTAR-DERECHA: la altura de x usa la de y, que todavía no se corrigió.
- Calcular $FB(v)$ en REBALANCEAR antes de $\text{ACTUALIZAR}(v)$.
- Recalcular la altura recorriendo el subárbol: cada operación pasa a costar $O(n)$.
- Mezclar convenciones: si $h(\text{NULO}) = -1$, una hoja tiene altura 0, no 1.

Complejidad

Operación	Tiempo	Rotaciones
Buscar, Mínimo, Máximo	$O(\lg n)$	—
Insertar	$O(\lg n)$	a lo más 2
Eliminar	$O(\lg n)$	$O(\lg n)$

Complejidad

Operación	Tiempo	Rotaciones
Buscar, Mínimo, Máximo	$O(\lg n)$	—
Insertar	$O(\lg n)$	a lo más 2
Eliminar	$O(\lg n)$	$O(\lg n)$

Espacio: $O(n)$, con un entero extra por nodo (*altura*). Insertar rota a lo más una vez (dos rotaciones si es un caso doble); eliminar puede rotar una vez por nivel, como en el ejemplo.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Cada nodo guarda su *altura*; ACTUALIZAR la recalcula en $O(1)$ con la de sus hijos.
- Las rotaciones reconectan tres punteros y actualizan dos nodos, de abajo hacia arriba.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Cada nodo guarda su *altura*; ACTUALIZAR la recalcula en $O(1)$ con la de sus hijos.
- Las rotaciones reconectan tres punteros y actualizan dos nodos, de abajo hacia arriba.
- INSERTAR y ELIMINAR son los del BST, recursivos, con REBALANCEAR al volver: insertar rota a lo más una vez; eliminar, a lo más una vez por nivel.
- Para probar la implementación: VERIFICAR revisa el orden, las alturas y $|FB| \leq 1$ en $O(n)$.

Logros de la sesión

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos mantener la altura de cada nodo y calcular su factor de balance en $O(1)$.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos mantener la altura de cada nodo y calcular su factor de balance en $O(1)$.
- Sabemos implementar las rotaciones y REBALANCEAR.

Logros de la sesión

- Sabemos mantener la altura de cada nodo y calcular su factor de balance en $O(1)$.
- Sabemos implementar las rotaciones y Rebalancear.
- Sabemos implementar Insertar y Eliminar en un AVL, y explicar cuántas rotaciones necesita cada uno.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos mantener la altura de cada nodo y calcular su factor de balance en $O(1)$.
- Sabemos implementar las rotaciones y Rebalancear.
- Sabemos implementar Insertar y Eliminar en un AVL, y explicar cuántas rotaciones necesita cada uno.
- Sabemos seguir a mano inserciones y eliminaciones, y verificar que el árbol siga siendo un AVL.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias